CASHCAST

Проект, технологии и защита

Единая техническая документация и методичка для питча

От выписки к решению

История операций → прогноз денег → объяснение дефицита → сравнение действий

Один счёт • RUB • 30 дней по умолчанию • API 1-180 дней

24 октября 2026 года, МСК

Техническая база: main e8f93fc
Pitch guide: main 56bebaf

Синтетические данные и исторические метрики обозначены. Вероятность нового кассового разрыва не подтверждена и не подменяется нулём.

Как читать документ

Часть I - бизнес-проблема, требования, демонстрация и защита. Часть II - полная техническая справка для разработки и эксплуатации. Часть III - происхождение материалов и границы проверки.

При противоречиях использована более поздняя техническая база e8f93fc. Цифры экспериментов и старые тестовые отчёты относятся к своим версиям. Новые продуктовые проверки при объединении не проводились.

Оглавление

I. Продукт, требования и защита **3**

I.1. Проблема и ценность для предпринимателя . . . 3
I.2. Требования кейса и фактическая готовность . . . 3
I.3. Какие числа можно защищать . . . 4
I.4. Сценарий демонстрации: от факта к решению . . . 5
I.5. Короткий питч и ответы экспертам . . . 5
I.6. До защиты: приоритеты и исторические проверки . . . 6
I.7. Структура финальной защиты и вопросы к экспертам . . . 7

II. Техническое устройство и эксплуатация **8**

II.1. Назначение и границы продукта . . . 8
II.2. Архитектура и стек . . . 8
II.3. Карта исходников . . . 9
II.4. Данные и инварианты . . . 10
II.5. Импорт и план-факт . . . 11
II.6. Прогноз и фоновые задания . . . 12
II.7. Сценарии, кассовые разрывы и финансовый пульс . . . 14
II.8. HTTP API . . . 14
II.9. Безопасность и ассистент . . . 16
II.10. Frontend . . . 17
II.11. Локальный запуск и настройки . . . 17
II.12. Проверки и эксплуатация . . . 19
II.13. Метрики и достоверность . . . 20
II.14. Ограничения и расхождения старых документов . . . 21

III. Источники и границы объединения **22**

IV. Полный альбом схем и проверка полноты

Архитектура; 11 сообщений расчёта; 16 таблиц и 33 FK; импорт, очередь, incremental, кабинет, сценарии, ассистент и Compose. Полная навигация - стр. 23.

I. Продукт, требования и защита

I.1. Проблема и ценность для предпринимателя

Прибыльная компания может остаться без денег в день обязательного платежа: поступления от клиентов приходят позже зарплаты, аренды и налогов. CashCast помогает совместить историю операций и будущие обязательства, увидеть дату и величину нехватки и заранее сравнить возможные действия.

Учебный пример, не результат датасета: на счёте 500 тыс. ₽, через неделю нужно заплатить 700 тыс. ₽, а поступление 400 тыс. ₽ ожидается через две недели. До поступления не хватает 200 тыс. ₽. Будущая выручка сама по себе не закрывает разницу сроков.

Основной пользователь - владелец малого бизнеса без отдельного финансового специалиста. Его решение: ускорить поступление, согласовать перенос допустимой закупки, изменить покупку или оценить финансирование. Категории, модель и график нужны для этого решения.

Реализована цепочка: CSV-выписка → категории и регулярности → планы и сверка оплат → ежедневный прогноз → объяснение дефицита → сценарии и варианты действий. Есть API, PostgreSQL, worker, история и кабинет. Главная незакрытая часть обязательного требования - достоверная вероятность нового кассового разрыва.

Основание: pitch guide, стр. 1-2; техническая документация, разделы 1, 6-7.

I.2. Требования кейса и фактическая готовность

Номера страниц исходного кейса ниже приведены по pitch guide. Сам 15-страничный PDF организаторов при объединении повторно не анализировался. «Есть» означает наличие реализации, а не подтверждение текущего демостенда или отсутствие дефектов.

Требование	Что реализовано	Граница и недоделки
Категоризация (кейс, стр. 4, 8)	Автоправила и ручная категория, сохраняемая при повторном импорте	Независимая accuracy на размеченных банковских данных не измерена; спорные назначения остаются
Регулярности (стр. 4)	Недельные, двухнедельные, месячные и квартальные повторы включаются в прогноз	Несколько договоров одного контрагента, пропущенные циклы и короткая история требуют доработки
Счета и налоги (стр. 4)	Ручные планы и CSV обязательств с суммой, датой и происхождением	Нет живого банка/1С и самостоятельного расчёта налогов; требование закрыто частично
Ежедневный остаток минимум на 30 дней (стр. 4, 8)	Пользовательский поток на 30 дней, API 1-180 дней, worker, сохранённый результат и история	Один счёт одной компании, RUB; нет консолидации и внутридневной ликвидности
Дата, сумма, вероятность разрыва (стр. 5)	Дата первого отрицательного p50, дефицит, эпизоды и состав минимума	Вероятность нового разрыва не доказана; default 2.0 выдаёт null, а не 0%
Понятный кабинет (стр. 5, 8)	Импорт, операции, планы, сверка, прогноз, сценарии, пульс и ассистент	Удобство для реальных предпринимателей отдельно не доказано
Модель и качество (стр. 8)	Baseline, seasonal, временная оценка, сравнение альтернатив и ограничения	Проверка подходов не означает достаточного качества всех задач

Требование	Что реализовано	Граница и недоделки
Контракты и дополнительные источники (стр. 6-7, 9)	Унифицированные обязательства, provenance, ручной ввод и CSV	Нет разбора договоров/отсрочек и подключения банка, 1С, ЭДО, касс, маркетплейсов в реальном времени
План-факт и обновление (стр. 6)	Остатки, просрочка, однозначное авто- и явное ручное сопоставление, новые прогнозы после действий	Полная история редактирования планов отсутствует; график расхождений требует отдельной приёмки
Четыре сценария (стр. 7)	Перенос события, снижение нерегулярного притока, дополнительный расход, рост регулярного потока	Снижение нерегулярного притока не равно снижению всей выручки; общее сравнение всех сценариев требует сверки с кейсом
Без полного пересчёта (стр. 9)	Incremental worker переиспользует совместимое состояние фактов/планов	API всё ещё готовит полный snapshot; при несовместимости - full; это не online-обучение
Тесты и безопасность (стр. 10)	Автотесты, браузерные сценарии, сессии, CSRF, роли и изоляция компаний	Production, пользовательская ценность, внешний аудит и зелёный CI текущего SHA здесь не подтверждены

По сравнению с более ранним pitch guide текущая техническая база дополнительно описывает отдельную карточку кассового разрыва, preview финансирования/переносов и ИИ-ассистента с подтверждением действий. Подробные границы этих функций - в части II.

Основание: pitch guide, стр. 3-4; техническая документация текущего main.

I.3. Какие числа можно защищать

Датасет организаторов - синтетический: 150 компаний, 222 420 операций, период 01.10.2024-30.09.2026. Это не результаты production-пилота. Экономия времени и бизнес-эффект у реальных клиентов не измерены.

Опубликованный эксперимент	Результат	Допустимый вывод
B4 / legacy 1.0: сентябрь, 122 окна × 30 дней	MAE остатка: baseline 2 775 628,60 ₽; seasonal w84 1 072 364,48 ₽. RMSE: 6 548 194,91 ₽ → 2 686 386,56 ₽	MAE ниже примерно на 61,4% относительно baseline в этом эксперименте, а не «точность 61,4%»
Выбор окна B4	84 дня выбраны на 517 validation-окнах	Выбор конфигурации отделён от описанного сентябрьского сравнения
B5: 268 общих окон, январь-март	Новые альтернативы не прошли заранее заданное улучшение MAE на 5%	Сохранена основа seasonal w84; сложность модели сама по себе не доказала пользу
B6: новые разрывы	Четыре положительных test-окна у трёх компаний; 2 обнаружения, 16 ложных тревог; precision 2/18, recall 2/4	Основания для надёжной продуктовой вероятности недостаточно
Описательный audit guarded 2.0	256 пригодных retrospective-окон / 7 680 дней; MAE 1 425 436,50 ₽	Другой набор наблюдений; нельзя напрямую сравнивать с MAE B4 как результат обновления

Сентябрь уже просмотрен до исправления проверки границы данных; повтор не является новым нетронутым holdout. Ошибки потоков высоки. На короткой истории B4 из четырёх компаний улучшения не показано. В B6 восемь из двенадцати положительных test-окон уже начинались с минуса; их нельзя выдавать за предсказанные

новые разрывы.

Защитимая формулировка: «Мы измерили ошибку остатка и сравнили подход с простым ориентиром. Отдельно проверили риск и обнаружили недостаток данных для надёжной вероятности нового разрыва. Поэтому не показываем недостоверный процент».

Ссылки на отчёты и оговорки к метрикам приведены в II.13. Все числа прочитаны из источников, без нового backtest.

I.4. Сценарий демонстрации: от факта к решению

1. Войти, выбрать один счёт, загрузить C001.csv, показать preview и явное подтверждение. Обозначить синтетическое происхождение данных. Не смешивать C001 и all_operations.csv: это повторяющиеся операции.
2. В операциях показать автоматическую и ручную категорию, а при повторном импорте - сохранение фактов и ручного выбора.
3. Открыть заранее проверенный 30-дневный прогноз: дата среза, график, предупреждения, дата/сумма дефицита при наличии. Если в исходной базе минуса нет, это явно обозначается.
4. Создать явно гипотетический дополнительный расход внутри горизонта и сравнить со сохранённой базой. Возникший от гипотезы разрыв не является фактом исходной выписки.
5. Показать карточку кассового разрыва: существующий/новый эпизод, восстановление, состав минимума. Preview переноса закупки или финансирования меняет только ответ; реальные планы сохраняются.
6. В пульсе показать подушку, действия, затраты финансирования и зависимость от клиентов. Источник, стоимость, допущения и долг за горизонтом раскрываются. Оценки пересекающейся экономии не суммируются.
7. На подготовленном совместимом примере показать incremental, исходный forecast_id и счётчики. Открыть старый прогноз в истории: результат не переписан.
8. При необходимости показать ассистента: текст и серверный график, предложение действия, отдельную кнопку подтверждения. Текстовое «да» само по себе не меняет финансы.

Перед выступлением нужно отдельно проверить фактический стенд: свежий Git не обновляет Docker автоматически. Сохранённые прогноз, сценарий и пульс стоит открыть заранее и подготовить резервную запись/скриншоты. Чужие localhost-порты из ночных отчётов не гарантируют наличие сервиса на этом ПК.

Это сценарий подготовки, а не отчёт о проведённой в этой задаче демонстрации. Основание: pitch guide, стр. 8; актуальные функции - часть II.

I.5. Короткий питч и ответы экспертам

Пример рассказа примерно на 90-120 секунд:

«Мы делаем CashCast для владельца малого бизнеса. Даже прибыльная компания может остаться без денег на зарплату или аренду: поступления и выплаты происходят в разные даты. Наш сервис помогает увидеть это заранее.

Уже работает путь от банковского CSV до ежедневного прогноза: категории, регулярные платежи, будущие обязательства, график и объяснения. Можно сравнить четыре негативных сценария, сверить план с фактом и сохранить историю. Финансовый пульс показывает гипотезы действий и их эффект на остаток. При новых операциях worker умеет частично обновлять расчёт.

На синтетических данных организаторов в опубликованном эксперименте legacy 1.0 сезонный подход снизил среднюю абсолютную ошибку остатка примерно на 61% относительно простого baseline. Это результат конкретной выборки, не процент точности текущего продукта. Надёжную вероятность нового разрыва мы пока

не подтвердили, поэтому не показываем ложную уверенность в процентах.

Следующий приоритет - сложные категории и регулярности, безопасная сверка оплат и отдельная проверка новых разрывов. От экспертов нам нужна обратная связь: какие предупреждения и действия действительно помогут предпринимателю и какой баланс ложных тревог и пропусков приемлем».

Вопрос	Короткий ответ
Чем лучше таблицы?	Автоматизирует разбор истории, прогноз, связь с планом и сравнение сценариев; сохраняет происхождение. Экономию времени ещё нужно измерить
Почему нет вероятности?	Её качество на новых разрывах не подтверждено. Дата и сумма по p50 доступны; null не означает нулевой риск
Какие интеграции работают?	Файловая загрузка выписок/обязательств и ручной ввод. Живых банковских и 1С-интеграций нет
Где нейросеть?	Прогноз строит объяснимая статистическая модель без обязательной LLM/GPU. Необязательный ассистент помогает работать с данными; числа рассчитывают серверные сервисы
Это готовый продукт?	Работающий прототип. Production-эксплуатация, пользовательская ценность и достоверность риска требуют дальнейшей проверки
Что нужно от экспертов?	Приоритет бизнес-сценариев, допустимые ложные тревоги/пропуски и доступ к размеченным новым разрывам

Не следует заявлять «готово на 75 баллов», «точность 61%», «вероятность доказана», «подключены банки» или «предотвращаем все разрывы». Корректно называть функцию, версию проверки и границу доказательств.

Основание: pitch guide, стр. 6, 9-10; рассказ согласован с текущей технической базой.

I.6. До защиты: приоритеты и исторические проверки

Сначала - достоверность ключевых результатов:

- Новые разрывы: дополнительные события при неотрицательном начальном остатке, независимые выбор/проверка порога, precision/recall, ложные тревоги и время предупреждения.
- Категории и регулярности: комиссии, налоги, кредиты, неоднозначные назначения, несколько договоров одного контрагента и пропущенные циклы.
- План-факт: проверить конфликты идентификаторов, частичные и альтернативные оплаты, отсутствие двойного распределения; довести график расхождений.

Затем - продуктовая приёмка: общее сравнение базы и четырёх сценариев, реалистичный внешний источник обязательств с измеренным вкладом, проверка интерфейса с предпринимателем и устойчивая демонстрация.

Исторические свидетельства из pitch guide, стр. 10:

Пакет / база	Приведённый результат	Как интерпретировать
Расширенная приёмка D, 19cd2377	89 проверок: 62 passed, 27 failed (D1: 23, D2: 3, D3: 1)	Не 27 независимых багов и не текущий полный тест main. Пакет не содержит исправлений. Упомянутые deliverables здесь независимо не проверялись

Пакет / база	Приведённый результат	Как интерпретировать
PR #50, incremental	636 backend passed до последней небольшой проверки checkpoint; затем 40 профильных; 15 frontend unit, lint/build; 12 E2E и повтор двух incremental E2E	Разные моменты проверки. Нельзя выдавать за единый прогон текущего SHA
Pulse на своей версии	592 backend и 10 E2E; C001 отдельно	Результаты не складываются с PR #50
GitHub Actions	В guide не заявлены зелёными; применялось согласованное исключение по квоте	Наличие workflow и старых локальных отчётов не определяет статус CI текущего commit

Из исходного guide сохранены проблемы для проверки, а не утверждение, что они до сих пор воспроизводятся или уже исправлены. До статуса «исправлено» нужна новая профильная проверка точной версии. В задаче объединения приложение и продуктовые тесты не запускались.

I.7. Структура финальной защиты и вопросы к экспертам

Шесть блоков презентации по пересказу кейса, стр. 12:

Блок	Материал
Проблема, решение, достижение цели	Пример несовпадения сроков денег и платежей; путь до решения
Работающий прототип	Импорт → прогноз → предупреждение → действие/сценарий, резервная запись
Источники данных	Синтетический датасет; ручные события и CSV обязательств отдельно от живых интеграций
Прогноз и качество	Компоненты, baseline/seasonal, метрики с единицами, объёмом выборки и ограничениями
Архитектура	Кабинет → API → PostgreSQL/worker → сохранённый результат, безопасность и snapshot
Дополнительные задачи и развитие	Сценарии, план-факт, пульс, incremental; незакрытое и следующий проверяемый этап

В guide названы пять критериев оценки: качество прогноза; корректность и заблаговременность выявления разрывов; дополнительные источники и их вклад; техническое качество; демонстрация и объяснение. Веса и формула на 100 баллов там не приведены; внутренние проценты готовности не являются оценкой жюри.

Полезные вопросы: что считать разрывом - минус одного счёта, нехватку на обязательства или дефицит до помощи владельца? Какая заблаговременность полезна и сколько ложных предупреждений допустимо? Какой источник будущих платежей важнее для пилота? Достаточен ли файловый обмен или ожидается живая интеграция?

Эти вопросы уточняют приоритеты, не заменяя явно заданных требований. Основание: pitch guide, стр. 11-12.

II. Техническое устройство и эксплуатация

II.1. Назначение и границы продукта

CashCast - сервис прогнозирования денежного потока малого бизнеса. Основная цепочка: загрузить выписку → проверить и подтвердить импорт → получить категории операций → рассчитать ежедневный остаток → учесть планы и оплаты → исследовать кассовый разрыв и сценарии действий.

Единица расчёта - **один банковский счёт одной компании, валюта RUB**. Система поддерживает несколько доступных компаний и счетов, но не консолидирует их в один прогноз. Стандартный горизонт - 30 дней; API принимает 1-180 дней. Дата среза `as_of` не может быть позже текущей даты UTC.

Реализованы выписки и обязательства CSV, ручные планы, план-факт, история расчётов, сценарии, финансовый пульс и необязательный текстовый ИИ-ассистент. Прямых банковских/1С-коннекторов, OCR и самостоятельного расчёта налогов нет. Налоговое обязательство вводится с уже известными датой и суммой.

II.2. Архитектура и стек

Это модульный монолит с отдельным процессом фонового расчёта и общей PostgreSQL. Очередь находится в БД; Redis, Celery и отдельный брокер не требуются.

A01. Полная схема перенесена на отдельную страницу

Все 7 компонентов, 6 связей и 3 подписи сохранены. Схема дана крупно, без сокращений.

Открыть полную схему: стр. 24. Альбом и карта полноты: стр. 23.

Слой	Технологии и закрепление
Интерфейс	React 18.3.1, TypeScript 5.6.3, Vite 6.4.4, TanStack Query 5.59.20
Графики	Собственный SVG-компонент `SpecChart`; зависимости Recharts в текущем frontend нет
Сервер	Python 3.12; FastAPI, Pydantic 2, SQLAlchemy 2, psycopg 3, Alembic
Backend runtime	Образ Python 3.12.15-slim, uv 0.13.5
Frontend build/runtime	Node 24.18.1-alpine → nginx 1.28.0-alpine
Хранилище	PostgreSQL 16.10-alpine; JSONB для снимков, деталей и служебных структур
Аналитика	Локальные CPU-вычисления; внешняя LLM не участвует в математике прогноза
Проверки	pytest, Ruff; ESLint, TypeScript, Node test runner, Playwright/Chromium

Диапазоны backend-зависимостей заданы в pyproject.toml, точные разрешённые версии - в uv.lock. Frontend закреплён в package.json и package-lock.json. Сборка использует `uv sync --frozen` и `npm ci`.

В Compose объявлены пять сервисов: `postgres`, одноразовый `migrate`, `api`, `worker`, `web`. API и worker запускаются после успешных миграций. Единственная публикуемая точка входа - `127.0.0.1:${WEB_PORT:-8080}`. БД и API остаются во внутренней сети. API и worker используют один backend-образ, но разные команды запуска.

Источники: Compose, backend Dockerfile, frontend Dockerfile, nginx, сборка API.

II.3. Карта исходников

Путь	Ответственность
`backend/app/core/`	Настройки, SQLAlchemy, аутентификация, доступ к компаниям, ошибки, денежные/ресурсные ограничения
`backend/app/core/models/`	ORM-модель и ограничения хранимых данных
`backend/app/api/v1/`	HTTP-ручки, права доступа и транзакционное связывание сервисов
`backend/app/api/schemas/`	Публичные Pydantic DTO, валидация и OpenAPI
`backend/app/modules/contracts.py`	Чистые DTO и интерфейсы между интеграцией и аналитикой, без ORM/HTTP
`backend/app/modules/imports/`	CSV, preview, идентификация операций, атомарное подтверждение
`backend/app/modules/categorization/`	Правила категорий и объяснения
`backend/app/modules/planning/`	Регулярности, календарь, источники и предложения сопоставления
`backend/app/modules/integration/`	Запись категорий и подтверждённых распределений план-факта
`backend/app/modules/forecasting/`	Реестр моделей, immutable snapshot, сценарии, guarded/legacy и incremental
`backend/app/modules/jobs/`	Очередь, обработчики, восстановление и поиск checkpoint
`backend/app/modules/cash_gaps/`	Эпизоды дефицита, разложение минимума, preview действий
`backend/app/modules/pulse/`	Финансовый пульс, аудит расходов и клиентские стресс-сценарии
`backend/app/modules/assistant/`	Провайдер, разрешённые инструменты, предложения и подтверждение
`backend/migrations/`	Alembic-миграции
`frontend/src/app/`	Оболочка, маршрутизация, выбранный счёт и контроллер прогноза
`frontend/src/features/`	Функциональные панели кабинета
`frontend/src/shared/`	API-клиент, денежное форматирование, права компании
`evaluation/`	Backtest, выбор метода, калибровка и агрегированные отчёты
`specs/`, `coordination/`	Контракты и исторические решения, приёмка и состояние разработки

II.4. Данные и инварианты

II.4.1. Денежные и временные правила

- Деньги в БД и публичном API - целые копейки, поля `*_minor`. Например, `125050` означает 1 250,50 ₽. Сумма операции неотрицательна; знак задаёт `in/out`.
- Аналитика может применять `float` внутри модели, с явным округлением на границе. Ввод проверяется на целочисленность и допустимый денежный диапазон.
- Остаток на конец дня хранится отдельно от операций. Повторение остатка в нескольких строках выписки не означает несколько денежных поступлений.
- `as_of` ограничивает доступную историю. Будущие факты не становятся признаками; обязательства могут лежать в будущем, если были известны на дату среза.
- Новые прогнозы сохраняют полный вход, `data_version`, `model_version` и `seed`. Старые записи без `input_snapshot` остаются читаемыми, но сценарий, пульс и карточка разрыва требуют нового расчёта базы. Новые импорты и правки не обновляют ранее завершённые результаты.

Это не полная историческая версия БД: операции фильтруются по банковской дате, планы - по `available_at`. Позднее импортированная операция с прошлой датой может изменить **новый** расчёт прежнего среза; уже сохранённый run остаётся неизменным.

Источники: ORM, контракты, денежная граница, кодирование snapshot.

II.4.2. Основные сущности

Таблицы	Назначение и ограничения
`app_user`, `user_session`	Пользователь, хеш пароля, серверная сессия; хранится хеш session token
`company`, `company_member`, `account`	Компания, членство с ролью `owner/viewer`, банковский счёт
`statement_import`	Состояние выписки; уникальность `(account_id, content_sha256)`
`bank_transaction`	Фактические операции; уникальность `(account_id, fingerprint)`, источник категории
`daily_balance`	Подтверждённый EOD-остаток; уникальность `(account_id, balance_date)`
`obligation_import`	Нормализованное preview обязательств; уникальный набор счёт/источник/хеш/срез
`planned_event`	План, первоначальная и оставшаяся сумма, статус, происхождение документа
`planned_event_match`	Кандидат или подтверждённая связь плана с фактической операцией
`forecast_run`	Вход `input_snapshot`, версии, статус, риск, объяснения и внутренний checkpoint
`forecast_point`	Дневные потоки и остатки p10/p50/p90 одного расчёта
`job`	Задание, ключ дедупликации, попытки, lease, время следующего запуска
`assistant_turn`	Идемпотентный ответ чата; уникальность `(user_id, request_id)`
`assistant_action`	Предложенное действие, аргументы, fingerprint, срок, решение и результат

Связи: компания → счета → операции/остатки/планы/прогнозы; план ↔ операция через распределение; прогноз → дневные точки. Сценарий ссылается на базовый `forecast_run`.

Последовательность миграций: `887980e46a3f` → `20261023_frozen_input` → `20261024_obligation_imports` → `20261024_assistant`. Текущий incremental checkpoint хранится в JSONB и отдельной таблицы не требует. Источник: миграции.

II.5. Импорт и план-факт

II.5.1. Выписки

Импорт разделён на **preview → явный commit**. Preview принимает multipart: `account_id`, строку JSON `mapping`, файл `file`. Соответствие колонок настраивается; пресет `participants-v1` поддерживает UTF-8 BOM, разделитель `;` и направления «Поступление/Списание». ИНН рассматривается как строковый идентификатор.

Приложение ограничивает размер файла 10 MiB по умолчанию. nginx также ограничивает весь HTTP body до `10m`, поэтому файл у самой границы может быть отклонён раньше с учётом multipart-обёртки. Изменение лимита требует согласованной настройки обоих слоёв.

Повтор того же файла возвращает прежний импорт; повтор подтверждения - сохранённую сводку первого commit. Изменение содержимого операции с прежним банковским ID даёт конфликт и откат всего нового файла, включая новые операции и остатки. Неоднозначная коллизия fingerprint также не должна молча удалять факт. Ручная категория сохраняется при повторном импорте.

Исходные bytes выписки сохраняются в `statement_import` в БД для повторного разбора при commit; резервная копия БД тоже содержит их. У обязательств исходный CSV не хранится, сохраняется нормализованное представление. Идентичные операции без внешнего ID могут быть неразличимы: нельзя обещать сохранение их кратности по одному совпадающему fingerprint. Commit импорта сам не ставит прогноз в очередь - кабинет делает отдельный POST после успеха.

Источники: HTTP импорта, парсер, сервис, идентичность, формат датасета.

II.5.2. Обязательства и сопоставление оплат

CSV обязательств имеет собственные preview/commit. Сохраняются исходная ссылка, происхождение, известная дата, налоговый режим при наличии и неизменяемое представление исходного документа. Идентичный повтор не сбрасывает оплату или отмену; изменённый документ с прежним внешним ключом даёт `409 SOURCE_ID_CONFLICT`.

Matcher выдаёт предложения, а интеграционный слой записывает подтверждённые связи и обновляет остатки в одной транзакции. Безопасное однозначное сопоставление можно применить автоматически; неоднозначное требует явного выбора операции и суммы. Проверяются остатки обеих сторон, поэтому одна оплата не расходуется дважды.

Исторический план-факт ограничен: полной истории всех редактирований планов нет. Неоднозначное восстановление состояния прошлого возвращает `409`, вместо использования сегодняшних сведений как известных в прошлом.

До matcher применяется ресурсный бюджет: активные планы × доступные операции не больше 100 000. Превышение отклоняется с `422 plan_fact_resource_limit`, без усечения кандидатов и частичных записей.

Источники: obligations, plan-fact интеграция, ручное подтверждение, matching.

II.6. Прогноз и фоновые задания

II.6.1. Жизненный цикл

A02. Полная схема перенесена на отдельную страницу

Все 4 участника и 11 сообщений сохранены раздельно, включая COMMIT и оба GET-запроса.

Открыть полную схему: стр. 25. Альбом и карта полноты: стр. 23.

Сборщик снимка учитывает подтверждённые распределения, категории, планы и остатки. Последний известный EOD переносится к `as_of` известными последующими потоками; расчётное происхождение начального остатка сопровождается предупреждениями. При сегодняшнем срезе создание прогноза также может применить безопасную сверку; исторический срез не записывает сегодняшние решения в прошлое.

Изменения счёта и построение снимка используют общую блокировку его строки. FastAPI-зависимость `get_db` выполняет commit/rollback до отправки ответа (`scope="function"`), чтобы сразу после `202` job был доступен другому запросу/worker.

Снимок ограничен 50 000 операций, 4 000 EOD-остатков и 2 000 известных планов. Превышение отклоняется явно, без молчаливого усечения истории. Эти бюджеты дополняют денежные ограничения и лимит потенциальных пар matcher.

Статусы задания: `pending → running → succeeded/failed`. Дедупликация возвращает тот же run/job при том же входе, модели, seed и сценарии. Это отдельный механизм, не доказательство incremental.

Worker использует `FOR UPDATE SKIP LOCKED`, lease и проверку владельца/попытки. По умолчанию lease - 300 секунд, максимум попыток - 3, опрос очереди - раз в секунду. Блокировка job удерживается при обработке: живой долгий расчёт не должен дублироваться из-за истечения lease. После смерти процесса возможны rollback и повторный захват. Уборщик переводит исчерпанные задания в `failed` и синхронизирует статус прогноза.

Явный повтор запроса owner может дать ещё три попытки прежнему run/job только для разрешённых временных ошибок; ошибки данных/модели этим способом не возобновляются. Гарантии транзакции касаются PostgreSQL, не произвольных внешних побочных эффектов.

Источники: forecasts API, snapshot, DB dependency, queue, handlers, recovery, бюджет snapshot, спецификация jobs.

II.6.2. Модели, качество и объяснения

Текущая модель по умолчанию - `seasonal-guarded-2.0-w84`. Реестр также сохраняет `baseline-repeat-week-0.1` и `seasonal-block-bootstrap-1.0-w28/w56/w84` для воспроизводимости старых расчётов. Публичный `ForecastRequest` не содержит выбора `model_version`; стандартный новый run закрепляет default сервера.

Математическая основа: категории → календарные регулярности → известные плановые потоки без двойного счёта → остаточные нерегулярные потоки → сезонная оценка с окном до 84 дней и block bootstrap → дневные остатки и эмпирические квантили. Версия 2.0 добавляет проверки качества истории и безопасные ограничения. При недостаточности данных p10/p90 могут совпасть с p50.

Категории определяются упорядоченными правилами по нормализованному назначению платежа: переводы, зарплата, налоги, комиссии, займы, аренда, продажи, закупки. Неизвестное остаётся `OTHER`; confidence эвристики не является измеренной точностью. Регулярности требуют минимум три разные даты, поддерживают weekly/biweekly и календарные monthly/quarterly с допусками по датам и суммам. Несколько обязательств одному контрагенту могут смешаться; годовая сезонность отдельно не моделируется.

При достаточной истории seasonal строит 200 парных недельных bootstrap-траекторий. Режимы quality - `plan_only`, `point_only`, `empirical`; несверка остатков ограничивает пригодную историю, в том числе регулярности. Свежая и согласованная история нужна для симуляции, но даже её достаточность не активирует числовую вероятность. Сумма ожидаемых потоков и p50 траекторий не обязаны совпадать: при разложении медианы отдельно учитывается техническая поправка.

Результат содержит `assumptions`, `data_warnings`, объяснения и происхождение. Даты первого минуса и максимальный дефицит описывают медианную траекторию. `probability_negative` относится к отдельному дню, `probability_any_negative` - ко всему горизонту, но **в текущей guarded-модели оба равны** `null`**,** `risk.available=false`. Это отсутствие подтверждённой вероятности, а не 0% риска. p10-p90 не являются гарантированным доверительным интервалом.

Источники: реестр, guarded, quality, категории, регулярности, спецификация модели.

II.6.3. Инкрементальное исполнение

Частичный расчёт уже реализован. Внутреннее состояние `ledger-update-v2` сохраняется в `forecast_run.details._incremental_state`, проверяется checksum и не выдаётся API. Ищутся последние восемь совместимых успешных базовых запусков той же компании, счёта, модели, seed и горизонта с датой не позже нового среза.

Worker обновляет категории изменённых операций, затронутые группы регулярностей, дневные агрегаты и bootstrap-траектории. При изменении допустимых блоков сетка может генерироваться заново; календарное сопоставление пересчитывается целиком. Новые факты, изменение планов и движение даты вперёд поддерживаются.

Полный расчёт применяется при первом запуске, отсутствии/повреждении checkpoint, несовместимых параметрах и для сценария. Идентичный запрос может вернуть прежний run. Оптимизирован worker; API всё ещё собирает полный snapshot, поэтому весь HTTP-запрос не имеет гарантированной сложности `O(delta)`.

Публичный блок `calculation` содержит `mode`, `reason`, `source_forecast_id`, счётчики переиспользованных/изменённых операций и сгенерированных траекторий. У старых результатов он может отсутствовать. Режим нужно проверять по этим полям, не по субъективной скорости ответа.

Источники: checkpoint repository, incremental engine, state, контракт.

II.7. Сценарии, кассовые разрывы и финансовый пульс

Обычный сценарий создаёт отдельный фоновой run из frozen-входа базы. API принимает 1-50 изменений: `delay_event`, `reduce_irregular_inflow`, `add_expense`, `increase_recurring`. Реальные операции и планы не меняются. Сравнение читает сохранённые результаты и проверяет совместимость входа, даты, горизонта, модели и seed; отдельный seed допустим для расчёта, но исключает парное сравнение с прежней базой.

Карточка **кассовых разрывов** читает успешный сохранённый прогноз. Учитываются начальный остаток на `as_of` и ежедневные p50. Ноль закрывает отрицательный эпизод. `existing` - минус уже на срезе с неизвестным настоящим началом; `new` - новый минус в горизонте. Длительность может быть нижней границей. Недостаток данных не превращается в утверждение об отсутствии риска.

`cash-gap-preview` принимает 1-3 гипотетических действия: перенос потока или финансирование с датой/суммой получения, возвратом и полной стоимостью. Перенос расхода разрешён только для закупки с явным согласием; налоги, зарплата, аренда и кредит защищены. Долг за пределами горизонта показывается отдельно. Preview изменяет медианную кривую в ответе: нет нового job, записи плана или пересчёта вероятностей. Им могут пользоваться owner и viewer своей компании.

Финансовый пульс объединяет анализ сохранённого остатка, варианты действий, аудит расходов, концентрацию B2B-клиентов и стресс задержки/потери поступлений. Основой служат frozen input и сохранённый p50, а не текущие live-таблицы. Стоимость финансирования и клиентские стрессы являются сценарными допущениями. Пересекающиеся оценки экономии нельзя складывать как независимые возможности. Синтетический peer-справочник датирован 30.09.2026 и отключён для более раннего среза. Продуктовая адаптация пульса не запускает отдельный архивный прогноз или модель привычек владельца. Гибкие расходы в её календаре определяются по распознанным регулярным закупкам; допустимость отсрочки требует согласования с контрагентом. Возможности standalone-архива не равны возможностям endpoint.

Источники: scenarios, comparison, cash gaps, контракт разрывов, pulse, контракт pulse. Сохранённая кривая и адаптация: frozen pulse.

II.8. HTTP API

Префикс всех путей таблицы - `/api/v1`. UUID и JSON-схемы, query-параметры, пагинация и ограничения полей определены в OpenAPI. На запущенном backend FastAPI также формирует `/openapi.json` и `/docs`; штатный nginx проксирует только `/api/`, поэтому эти корневые адреса не доступны через web без дополнительной настройки.

Метод	Путь без префикса	Назначение
GET	`/health`	Проверка API и `SELECT 1` в БД
POST	`/auth/login`	Вход, cookie и CSRF
GET	`/auth/me`	Пользователь, CSRF, доступные компании и роли
POST	`/auth/logout`	Удаление серверной сессии
GET	`/companies/{company_id}/accounts`	Счета компании
GET	`/import-presets`	Поддерживаемые настройки колонок CSV
POST	`/companies/{company_id}/imports`	Preview выписки
POST	`/imports/{import_id}/commit`	Подтверждение выписки

Метод	Путь без префикса	Назначение
POST	`/companies/{company_id}/obligation-imports`	Preview обязательств
POST	`/obligation-imports/{import_id}/commit`	Подтверждение обязательств
GET	`/companies/{company_id}/transactions`	Операции с пагинацией
PATCH	`/transactions/{transaction_id}/category`	Ручная категория
GET	`/companies/{company_id}/planned-events`	Планы
POST	`/companies/{company_id}/planned-events`	Создание плана
PATCH	`/planned-events/{event_id}`	Изменение или отмена плана
GET	`/companies/{company_id}/plan-fact`	Просмотр сверки и предложений
POST	`/companies/{company_id}/plan-fact`	Применение безопасных совпадений
POST	`/companies/{company_id}/plan-fact/confirm`	Подтверждение конкретного распределения
POST	`/companies/{company_id}/forecasts`	Запрос прогноза, ответ `202`
GET	`/companies/{company_id}/forecasts`	История расчётов выбранного счёта
GET	`/jobs/{job_id}`	Статус фонового задания
GET	`/forecasts/{forecast_id}`	Сохранённый прогноз
GET	`/forecasts/{forecast_id}/explanations`	Компоненты и происхождение
POST	`/forecasts/{forecast_id}/scenarios`	Отдельный сценарный расчёт
GET	`/forecasts/{forecast_id}/comparison`	Сравнение с базой
GET	`/forecasts/{forecast_id}/pulse`	Финансовый пульс
GET	`/forecasts/{forecast_id}/cash-gaps`	Эпизоды дефицита и состав минимума
POST	`/forecasts/{forecast_id}/cash-gap-preview`	Вычисление гипотетических действий
GET	`/assistant/status`	Доступность провайдера и модель
POST	`/companies/{company_id}/assistant/chat`	Чат и разрешённые инструменты
POST	`/companies/{company_id}/accounts/{account_id}/assistant/actions/{action_id}`	Подтвердить/отменить предложение
POST	`/companies/{company_id}/accounts/{account_id}/assistant/uploads`	Preview файла в чате

Пример тела запроса прогноза; UUID иллюстративный, нужно подставить доступный счёт:

```
{
  "account_id": "00000000-0000-4000-8000-000000000001",
  "as_of": "2026-09-30",
  "horizon_days": 30,
  "seed": 42
}
```

Ответ `202` содержит `job_id`, `forecast_run_id`, `status` и заголовок `Location: /api/v1/jobs/{job_id}`. После `succeeded` клиент читает результат. Открытие истории не запускает новый расчёт. Операции и история прогнозов используют `items`/`next_cursor`; список планов возвращается без курсорной пагинации. Курсор не заменяет серверную проверку компании и счёта. CRUD пользователей, ролей и банковских счетов публичным API не предоставляется.

Единый формат ошибки приложения:

```
{
  "error": {
    "code": "validation_error",
    "message": "Некорректные данные запроса",
    "field_errors": [],
    "request_id": "идентификатор-запроса"
  }
}
```

Клиент должен опираться на `code`, а не текст. Типичные статусы: `401` - нет сессии, `403` - CSRF/нет права записи, `404` - объект недоступен, `409` - конфликт состояния, `413` - размер файла, `422` - некорректный ввод или ресурсная граница. Ошибки, сформированные самим nginx, могут иметь другой формат.

II.9. Безопасность и ассистент

Сессия хранится на сервере, токен передаётся в `cc_session` (`HttpOnly`, `SameSite=Lax`). В БД хранится хеш токена; стандартный срок сессии - 12 часов. CSRF-токен выдаётся в `cc_csrf` и JSON login/me. Сервер сравнивает `X-CSRF-Token` с токеном серверной сессии; отдельного сравнения присланной CSRF-cookie с заголовком нет. Вход - отдельный начальный запрос; остальные защищённые POST/PATCH требуют CSRF.

Роль определяется членством в **целевой** компании. `owner` изменяет данные, `viewer` читает, включая вычислительный preview без финансовой записи. Недоступная компания/объект возвращает `404`; наличие UUID не даёт доступа. Скрытие кнопок интерфейсом дополняет серверные ограничения.

Ассистент использует OpenRouter; настроенная по умолчанию модель - `deepseek/deepseek-v4.1-flash-1017`. Без ключа отключается текстовый провайдер, остальные функции кабинета продолжают работать. Ключ получает только API. Это значение конфигурации проекта, доступность провайдера в ходе анализа не проверялась.

Чат может читать данные через ограниченный gateway. Графики/таблицы берутся из серверных данных, а не исполняемого кода модели. Финансовое изменение проходит через `propose_*` → карточку → отдельную кнопку подтверждения. Текстовое «да» не является подтверждением; у модели нет инструмента подтверждения.

Предложение привязано к пользователю, компании и счёту, имеет fingerprint состояния и срок 15 минут. Подтверждение проверяет owner, CSRF, срок и актуальность под блокировкой счёта; изменение и итог предложения сохраняются атомарно. Повтор подтверждения возвращает сохранённый результат. Viewer не может предлагать или подтверждать изменения.

Модель получает сообщения и подготовленные показатели/результаты разрешённых инструментов. CSV-вложение само по себе не отправляет исходный файл провайдеру. Секреты в чат вводить не следует. Очистка видимого чата не удаляет серверный журнал ответов и предложений. Лимит по умолчанию - 100 ходов на пользователя за UTC-день.

Приложение не должно логировать содержимое выписок, назначения, пароли и токены. Обработчик неожиданных ошибок пишет тип и стек без `str(exception)` и исходного ввода. `request_id` позволяет сопоставлять запросы и ошибки.

Источники: auth, company access, ошибки, assistant service, actions, спецификация ассистента.

II.10. Frontend

Текущий интерфейс - SPA с собственной pathname-маршрутизацией и History API:

URL	Раздел
`/overview`	Обзор выбранного счёта
`/operations`	Банковские операции и ручные категории
`/import`	Preview и подтверждение выписки
`/payments`	Планы и обязательства
`/reconciliation`	План-факт и подтверждение оплат
`/forecast`	Прогноз, объяснения, история и кассовые разрывы
`/scenarios`	Создание сценариев и сравнение
`/pulse`	Финансовый пульс
`/assistant`	ИИ-ассистент

Неизвестный путь нормализуется в `/overview`; nginx поддерживает SPA fallback. TanStack Query хранит серверное состояние. Переключение пользователя очищает данные прежней сессии; выбранный счёт и права передаются через контексты. API-клиент отправляет cookie и CSRF для изменяющих запросов.

После успешного импорта, изменения плана/категории или сверки кабинет автоматически запрашивает новый базовый прогноз и ждёт worker. Для viewer автоматическая запись не выполняется. Reload, открытие истории и смена даты сами по себе не означают POST. Изменения через сторонний API-клиент не вызывают серверную push-подписку в браузере. Действия ассистента имеют собственный путь явного подтверждения.

После импорта выписки срез берётся по последней дате файла; после изменения обязательств/планов/сверки - сегодняшний UTC-день; категория сохраняет выбранный срез. Job опрашивается каждые 1500 мс до конечного статуса. Запись через ассистента помечает график устаревшим без скрытого нового прогноза; подтверждённый запрос прогноза открывает уже созданное задание.

Источники: router, AppShell, forecast controller, API-клиент, SVG-графики.

II.11. Локальный запуск и настройки

II.11.1. Docker Compose

Нужны Git, доступ к приватному репозиторию и Docker Compose v2; на Windows - Docker Desktop с Linux-контейнерами. Python и Node на хосте для этого режима не нужны. Команды выполняются из корня проекта; существующий `infra/.env` не перезаписывается:

```
if (-not (Test-Path -LiteralPath 'infra/.env')) {
    Copy-Item -LiteralPath 'infra/.env.example' -Destination 'infra/.env'
}
docker compose --env-file infra/.env -f infra/docker-compose.yml up -d --build
docker compose --env-file infra/.env -f infra/docker-compose.yml ps -a
curl.exe --fail http://127.0.0.1:8080/api/v1/health
```

Открыть `http://localhost:8080` (или выбранный `WEB_PORT`). Локальные демонстрационные учётные данные описаны в README и env-примере. `migrate` должен завершиться с кодом 0, остальные четыре сервиса - работать. Seed создаёт отсутствующего demo-пользователя, но не меняет пароль существующего при очередной правке env.

Переменная	Назначение / стандартное значение
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`	Учётная запись и БД контейнера
`CC_DATABASE_URL`	SQLAlchemy URL с драйвером `postgresql+psycopg`
`WEB_PORT`	Локальный web-порт, `8080`
`CC_APP_ENV`	Режим, `dev`
`CC_COOKIE_SECURE`	`false` для локального HTTP; для HTTPS требуется `true`
`CC_SEED_DEMO`, `CC_DEMO_EMAIL`, `CC_DEMO_PASSWORD`	Создание демо-учётной записи при миграциях
`CC_OPENROUTER_API_KEY`	Необязательный секрет ассистента, только API
`CC_ASSISTANT_MODEL`	ID текстовой модели
`CC_ASSISTANT_DAILY_TURNS`	Лимит, `100`
`CC_TEST_DATABASE_URL`	Отдельная уничтожаемая БД тестов с суффиксом `_test`
`CC_DEV_API_TARGET`	Proxy target Vite, по умолчанию `http://127.0.0.1:8080`
`CC_E2E_BASE_URL`	Адрес отдельного E2E-стенда, по умолчанию `http://127.0.0.1:18081`

Pydantic Settings читает окружение с префиксом `CC_`, не подхватывает `.env` автоматически. Compose передаёт в контейнеры только перечисленные в YAML параметры; добавления произвольного `CC_*` в `infra/.env` недостаточно, если параметр не проброшен в нужный сервис.

II.11.2. Разработка

Frontend поверх работающего API/Compose, из каталога `frontend`:

```
npm ci
# Только если API-шлюз работает не на 8080:
# $env:CC_DEV_API_TARGET = 'http://127.0.0.1:18086'
npm run dev
```

Vite слушает `127.0.0.1:5174`, проксирует `/api`; занятый порт вызывает ошибку, автоматического выбора другого порта нет. Требуется Node 24.

Backend, из каталога `backend`, с отдельной локальной dev-БД:

```
uv sync --frozen
$env:CC_DATABASE_URL = 'postgresql+psycopg://USER:PASSWORD@127.0.0.1:5432/cashcast_dev'
uv run alembic upgrade head
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000
```

В другом терминале с тем же CC_DATABASE_URL: `uv run python -m app.worker`. Для Vite в этом режиме задаётся CC_DEV_API_TARGET=http://127.0.0.1:8000. Демо-пользователь при прямом запуске uvicorn автоматически не создаётся. Для демонстрационных пользователя, компании и счёта после миграций выполните `uv run python -m app.scripts.seed` с тем же URL БД и нужными CC_DEMO_*. Это явная запись демонстрационных данных в выбранную dev-БД.

Источники: config, Vite config, migrate/seed, seed.

II.12. Проверки и эксплуатация

II.12.1. Проверки разработчика

В этом анализе команды ниже **не выполнялись**; это инструкция воспроизведения. Полный pytest очищает таблицы через TRUNCATE ... CASCADE: использовать только отдельную уничтожаемую PostgreSQL БД с именем *_test, никогда БД кабинета. Ограничение проверяется fixtures.

Из backend:

```
uv sync --frozen
uv run pytest tests/analytics -q
uv run ruff check . ../evaluation
# Полный набор - только после создания отдельной БД:
$env:CC_TEST_DATABASE_URL = 'postgresql+psycopg://USER:PASSWORD@127.0.0.1:55439/cashcast_test'
uv run pytest -q
# Отдельная проверка миграций - только на той же тестовой БД:
$env:CC_DATABASE_URL = $env:CC_TEST_DATABASE_URL
uv run alembic upgrade head
uv run alembic check
uv run python -m app.scripts.export_openapi
```

Последняя команда перегенерирует specs/openapi.json; изменения следует проверить через diff. `alembic upgrade head`/`alembic check` используют CC_DATABASE_URL, поэтому для отдельной проверки миграций нужно явно направить и его на тестовую БД. Чистые tests/analytics не требуют PostgreSQL.

Из frontend:

```
npm ci
npm run lint
npm run typecheck
npm run test:unit
npm run build
```

E2E пишут синтетические данные: запускать на отдельном Compose project/volume, задав CC_E2E_BASE_URL, затем `npx playwright install chromium` и `npm run test:e2e`. Smoke-скрипты с `--write-demo-data` также изменяют выбранный стенд.

CI описывает три группы: backend с PostgreSQL, миграциями и проверкой OpenAPI; frontend; чистый Compose startup и Chromium E2E, включая Vite/StrictMode. Наличие workflow не подтверждает зелёный статус текущего SHA. Ночное исключение по квоте Actions хранится в решении; оно не заменяет локальные проверки и ревью.

II.12.2. Эксплуатационные свойства

- Данные сохраняются в volume pgdata. Обычный `docker compose down` не удаляет volume; `down -v` уничтожает хранимые данные и не подходит для обычной остановки.
- Резервная копия - PostgreSQL dump; проверять восстановление следует в новую пустую БД. Старые отчёты о restore не доказывают пригодность новой копии.

- nginx обновляет адрес upstream API через Docker DNS; при пересоздании API не требуется вручную закреплять прежний IP.
- `/api/v1/health` проверяет БД, но не гарантирует, что worker исполняет очередь. Состояние расчёта проверяется отдельно по job.
- Compose настроен как локальный стенд. Публичный TLS-деплой, HA, внешняя система мониторинга и регламент автоматических резервных копий здесь не настроены.

Симптом	Что проверить
`401` после простоя	Истёкшая cookie-сессия; повторный вход
`403` при записи	Роль целевой компании, cookie и `X-CSRF-Token`
Долго `pending/running`	Worker, подключение к БД, lease/попытки и `error_code`
Нет текстового ответа ассистента	`/assistant/status`, наличие ключа только у API, лимит и доступность провайдера
План не попал в прошлый прогноз	Время доступности на `as_of`; старый frozen результат не меняется
Вместо вероятности пусто	Политика guarded 2.0; это ожидаемое `null`
Ответ `413`	Лимит API и nginx с учётом multipart

II.13. Метрики и достоверность

Метрики ниже **прочитаны из опубликованных отчётов репозитория**, не пересчитаны в ходе подготовки документа. Они относятся к указанным экспериментам и выборкам синтетического датасета организаторов, а не к подтверждённым production-результатам.

Источник	Результат и интерпретация
B4 REPORT, holdout.json	Сентябрьская оценка legacy 1.0: 122 folds; MAE остатка seasonal 1 072 364,48 ₽ против baseline 2 775 628,60 ₽. Это абсолютная ошибка, не процент точности текущего приложения
B5 REPORT	268 общих folds за январь-март. Новые point-методы не прошли заданный порог улучшения 5%; сохранена основа seasonal w84
B6 REPORT, calibration-v2.json	Для новых разрывов на test всего четыре положительных fold у трёх компаний: 2 обнаружения и 16 ложных тревог, precision 2/18, recall 2/4. Этого недостаточно для продуктовой вероятности

После просмотра сентябрьских результатов их повторная обработка не образует новый независимый holdout. Дни/folds одной компании не являются независимыми компаниями. В датасете нет достаточной эталонной разметки категорий и сопоставления обязательств, поэтому точность этих модулей не заявляется. Confidence правила не равен accuracy. Синтетические smoke/E2E подтверждают интеграцию, а не качество прогноза.

Backtest и калибровка воспроизводятся скриптами `evaluation/backtest.py`, `select_v2.py`, `calibrate_v2.py`; команды и границы экспериментов приведены в отчётах. Исходный ZIP передаётся отдельно и не входит в Git. Не следует коммитить выписки, `.env`, дампы, токены, окружения и частные датасеты.

II.14. Ограничения и расхождения старых документов

1. **Вероятность нового разрыва недоступна.** Даты/суммы p50 доступны, проценты отключены. Подтверждённая независимая временная оценка нужна до их активации.
2. **Остатки дневные.** Внутридневной недостаток ликвидности из этих данных не определяется.
3. **История планов неполная.** Прошлое состояние не всегда можно восстановить; действуют явные guards и ошибки конфликта.
4. **Incremental относится к worker.** Snapshot на HTTP-стороне всё ещё полный; ускорение всего запроса и production SLA не доказаны.
5. **Сценарии и рекомендации - гипотезы.** Они не меняют настоящие планы без отдельного разрешённого действия; preview не доказывает устранение риска после горизонта.
6. **CSV mapping и повторы имеют ограничения.** Историческая задача повторного preview уже ошибочного файла с другим mapping отмечена как F2 BLOCKED; не следует считать её решённой только по общему описанию импорта.
7. **Ночные статусы описывают свою дату.** `coordination/REQUIREMENTS.md` ещё говорит, что incremental не реализован; текущий код и `specs/incremental-forecast.md` уже v2. Начало README описывает прежний минимальный интерфейс, а строка стека упоминает Recharts; текущая SPA имеет девять разделов и собственный SVG-график. Начальные формулировки B6 о старом default также исторические: активен guarded 2.0.
8. **Текущий SHA требует собственных проверок.** Старые количества пройденных тестов, локальные порты приёмки и READY отдельных пакетов не являются результатом проверки этого документа или текущего запуска приложения.

Правила дальнейшей работы: AGENTS.md и PROTOCOL. Изменения общих контрактов/миграций согласуются по этому протоколу; `main` обновляется через PR с человеческим одобрением.

III. Источники и границы объединения

Техническая часть подготовлена по docs/technical-documentation.md: main e8f93fc5c9db4dc8f7d3535a0d8b9dde47505615, 24.10.2026. В ней 32 операции API, модель данных, расчётный конвейер, интерфейс, запуск и ограничения. Ссылки в PDF ведут на зафиксированную версию приватного репозитория и требуют доступа к GitHub.

Презентационная часть основана на cashcast-pitch-guide.pdf, 12 страниц, 24.10.2026, база main 56bebaff6d9e9b6f4539b4b9b05ae9327edafaad. Сохранены постановка проблемы, требования и границы готовности, метрики с оговорками, демонстрация, короткий питч, вопросы, приоритеты и историческая приёмка.

Повторы архитектуры и алгоритма сведены в технической части. Устаревшее упоминание Recharts заменено фактическим собственным SVG-графиком; техническая версия и текущие функции взяты из более поздней документации. Рекомендации исходного guide трактуются как материал для подготовки к защите, а не как команды на запуск или изменение проекта.

Исходный PDF кейса, внешний пакет приёмки D и текущий статус GitHub Actions в задаче объединения отдельно не проверялись. Результаты прежних запусков не выданы за новые тесты. При подготовке этого PDF приложение, база и продуктовые тесты не запускались; проверялись сборка, содержание и вёрстка документа.

IV. Проверка полноты и навигация

Исходники: docs/technical-documentation.md; specs/architecture.md; ORM; API, jobs, forecasting, assistant, frontend и Compose.

В прежнем PDF обе исходные схемы были сокращены вручную. Архитектура потеряла подписи связей и части названий; диаграмма прогноза объединила отдельные действия. Сокращённые рисунки удалены из основной части. Их места теперь ведут к полноразмерным схемам A01 и A02.

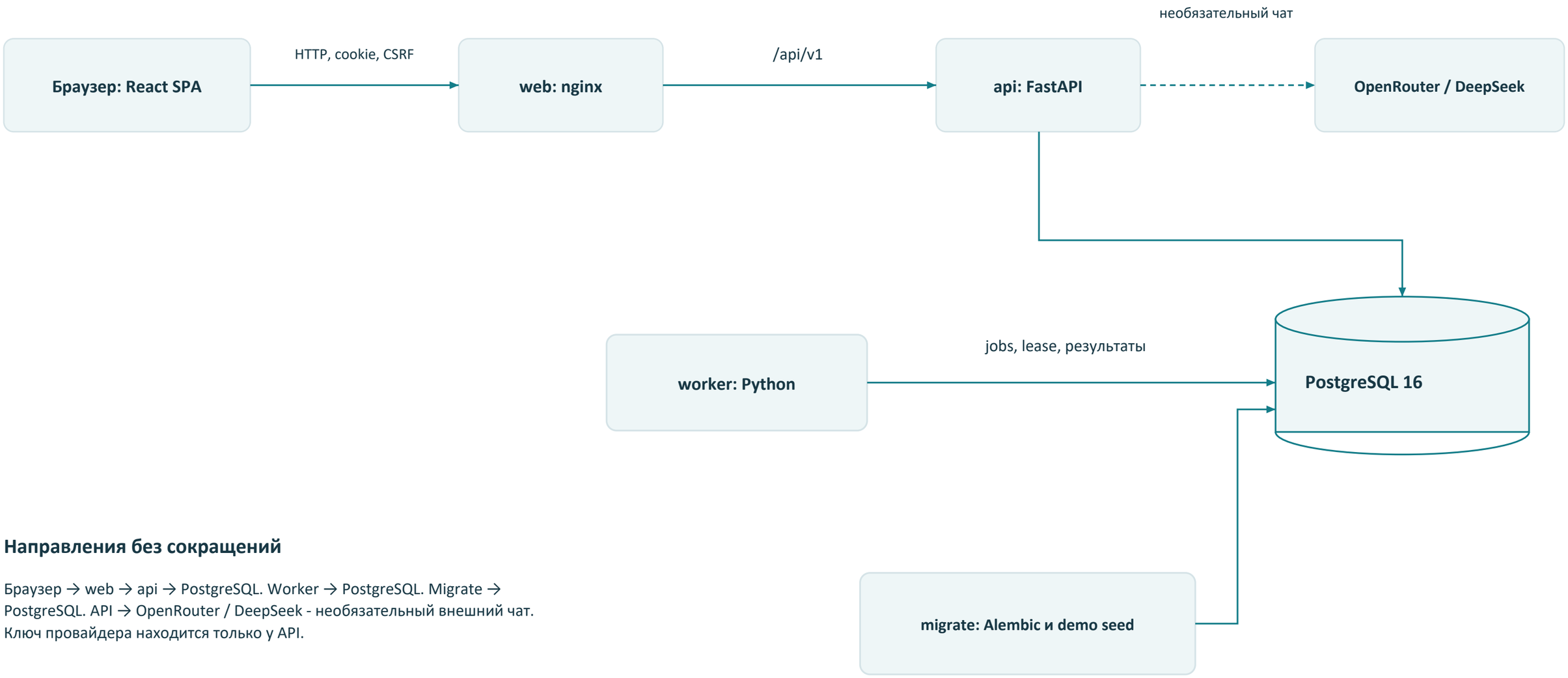
Проверяемый оригинал	Что сохранено в новой версии	Полная схема
flowchart LR / архитектура	7 компонентов; 6 направленных связей; все 3 подписи связей	A01, стр. 24
sequenceDiagram / прогноз	4 участника; 11 раздельных сообщений; полный текст каждого сообщения	A02, стр. 25
Модель данных	16 таблиц; 33 FK из AST; nullable и ondelete каждого FK	A03, стр. 26

Схема	Содержание	Страница
A01	Архитектура: все компоненты, направления и подписи	24
A02	Жизненный цикл прогноза: все 11 сообщений	25
A03	Данные: все 16 таблиц и все внешние ключи	26
A03.1	Полный реестр внешних ключей	27
A03.2	Полный реестр внешних ключей	28
A04	Выписки, обязательства и план-факт	29
A05	Прогноз: полный расчёт и incremental	30
A06	Очередь: статусы, повтор и восстановление	31
A07	Кабинет: все маршруты и обновления	32
A08	Сценарии, кассовые разрывы и пульс	33
A09	Ассистент: чтение, предложение и подтверждение	34
A10	Compose: запуск, сеть и хранение	35

Альбом дополняет техническую документацию на фиксированном commit. После fetch origin/main уже содержит 8168ec9 (без изменения топологии этих схем); база основного текста сохранена, чтобы не смешивать версии. Проверялась полнота представления, без запуска приложения и тестовой БД.

A01. Архитектура: все компоненты, направления и подписи

Источник: docs/technical-documentation.md, flowchart LR; полный состав соответствует присланному скриншоту.



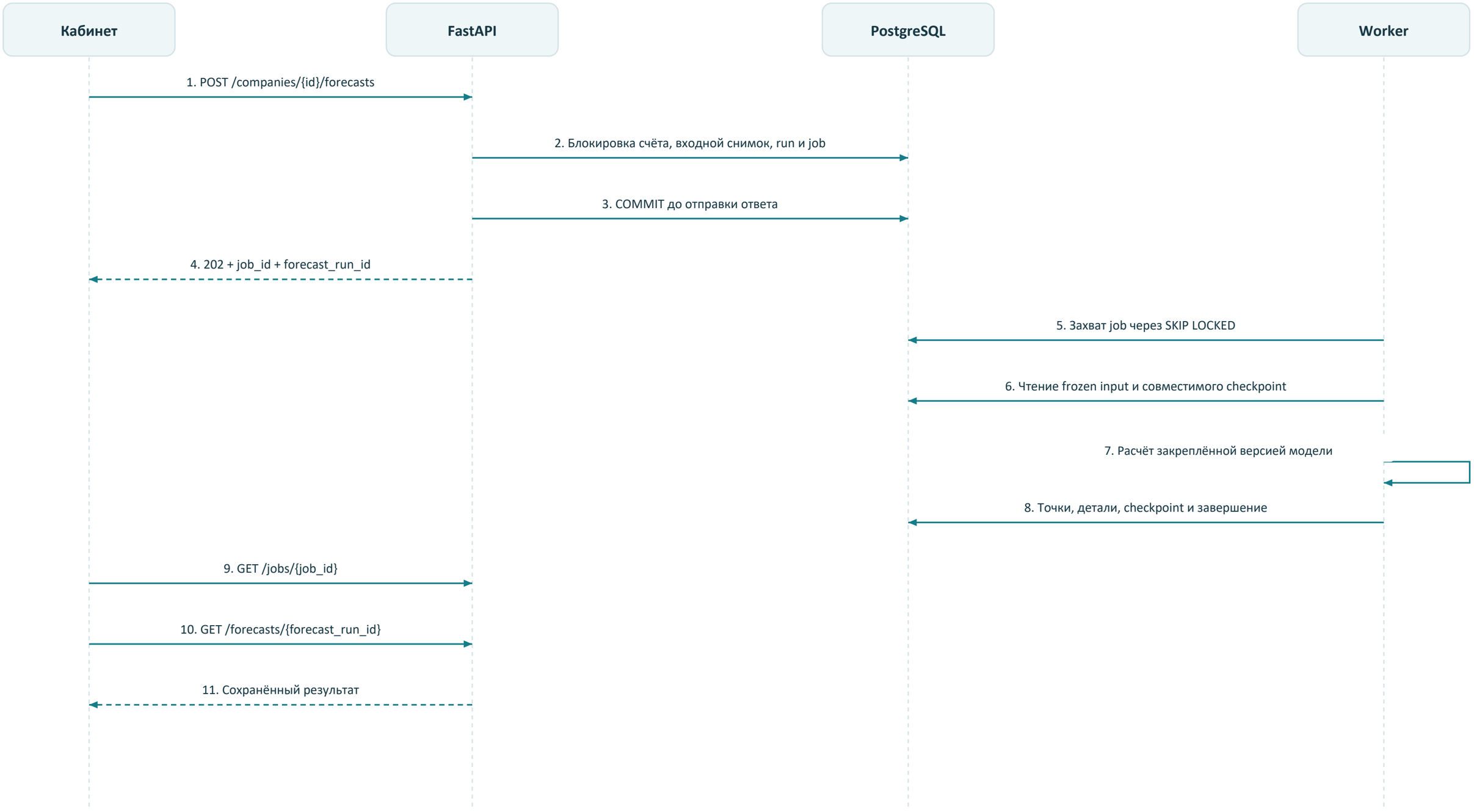
Направления без сокращений

Браузер → web → api → PostgreSQL. Worker → PostgreSQL. Migrate → PostgreSQL. API → OpenRouter / DeepSeek - необязательный внешний чат. Ключ провайдера находится только у API.

Очередь и результаты находятся в PostgreSQL; прямой вызов worker из API не изображается как реальная сетевая связь. API ставит job в БД, worker читает очередь. Дополнение запуска и портов - A10.

A02. Жизненный цикл прогноза: все 11 сообщений

Источник: исходный sequenceDiagram; COMMIT, GET job и GET forecast показаны отдельными сообщениями.



Порядок 1-11 и полный текст сохранены. Пунктир обозначает ответы. Дополнительная детализация retry/lease - A06; ветвление full/incremental - A05.

A03. Все таблицы и все внешние ключи

Источник: AST backend/app/core/models/*.py. Строка - дочерняя таблица, столбец - родительская. Fxx - уникальный FK; все Fxx раскрыты на следующих страницах.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
01. account						01										
02. app_user																
03. assistant_action	02	04				03										
04. assistant_turn	05	07				06										
05. bank_transaction	08					09									10	
06. company																
07. company_member		12				11										
08. daily_balance	13					14									15	
09. forecast_point										16						
10. forecast_run	17	20				19				18						
11. job						21										
12. obligation_import	22	24				23										
13. planned_event	25	27				26										
14. planned_event_match					29								28			
15. statement_import	30	32				31										
16. user_session		33														

Как читать

16 таблиц
33 внешних ключей

Число в клетке - номер Fxx из реестра. Направление: source.column → target.column. Два FK одной пары таблиц не объединяются.

Матрица включает self-FK forecast_run.base_forecast_id.

Связь job → forecast_run находится в JSON payload и не является FK. Checkpoint и provenance также не превращаются в выдуманные таблицы.

Первичные ключи

company_member: (company_id, user_id); forecast_point: (forecast_run_id, point_date). Остальные 14 таблиц: id. Иные поля, уникальности и проверки описаны в основной модели данных; внешние ключи перечислены полностью.

A03.1. Полный реестр FK: 1-19

Никакие FK не опущены. Nullable - из ORM-аннотации; ON DELETE - из ForeignKey. В реестре только реальные ограничения БД.

ID	Дочернее поле	Родительское поле	Nullable	ON DELETE
F01	account.company_id	company.id	нет	CASCADE
F02	assistant_action.account_id	account.id	нет	CASCADE
F03	assistant_action.company_id	company.id	нет	CASCADE
F04	assistant_action.user_id	app_user.id	нет	CASCADE
F05	assistant_turn.account_id	account.id	нет	CASCADE
F06	assistant_turn.company_id	company.id	нет	CASCADE
F07	assistant_turn.user_id	app_user.id	нет	CASCADE
F08	bank_transaction.account_id	account.id	нет	CASCADE
F09	bank_transaction.company_id	company.id	нет	CASCADE
F10	bank_transaction.import_id	statement_import.id	да	SET NULL
F11	company_member.company_id	company.id	нет	CASCADE
F12	company_member.user_id	app_user.id	нет	CASCADE
F13	daily_balance.account_id	account.id	нет	CASCADE
F14	daily_balance.company_id	company.id	нет	CASCADE
F15	daily_balance.import_id	statement_import.id	да	SET NULL
F16	forecast_point.forecast_run_id	forecast_run.id	нет	CASCADE
F17	forecast_run.account_id	account.id	нет	CASCADE
F18	forecast_run.base_forecast_id	forecast_run.id	да	CASCADE
F19	forecast_run.company_id	company.id	нет	CASCADE

Служебные связи вне FK: job.payload.forecast_run_id; forecast_run.details._incremental_state; ссылки provenance/source_ref. Они описаны отдельно, не подменяют ссылочную целостность PostgreSQL.

A03.2. Полный реестр FK: 20-33

Никакие FK не опущены. Nullable - из ORM-аннотации; ON DELETE - из ForeignKey. В реестре только реальные ограничения БД.

ID	Дочернее поле	Родительское поле	Nullable	ON DELETE
F20	forecast_run.created_by	app_user.id	да	SET NULL
F21	job.company_id	company.id	да	CASCADE
F22	obligation_import.account_id	account.id	нет	CASCADE
F23	obligation_import.company_id	company.id	нет	CASCADE
F24	obligation_import.uploaded_by	app_user.id	да	SET NULL
F25	planned_event.account_id	account.id	да	SET NULL
F26	planned_event.company_id	company.id	нет	CASCADE
F27	planned_event.created_by	app_user.id	да	SET NULL
F28	planned_event_match.planned_event_id	planned_event.id	нет	CASCADE
F29	planned_event_match.transaction_id	bank_transaction.id	нет	CASCADE
F30	statement_import.account_id	account.id	нет	CASCADE
F31	statement_import.company_id	company.id	нет	CASCADE
F32	statement_import.uploaded_by	app_user.id	да	SET NULL
F33	user_session.user_id	app_user.id	нет	CASCADE

Служебные связи вне FK: job.payload.forecast_run_id; forecast_run.details._incremental_state; ссылки provenance/source_ref. Они описаны отдельно, не подменяют ссылочную целостность PostgreSQL.

A04. Выписки, обязательства и план-факт

Источники: modules/imports; modules/planning/sources.py и matching.py; modules/integration/plan_fact.py и manual_confirmation.py.

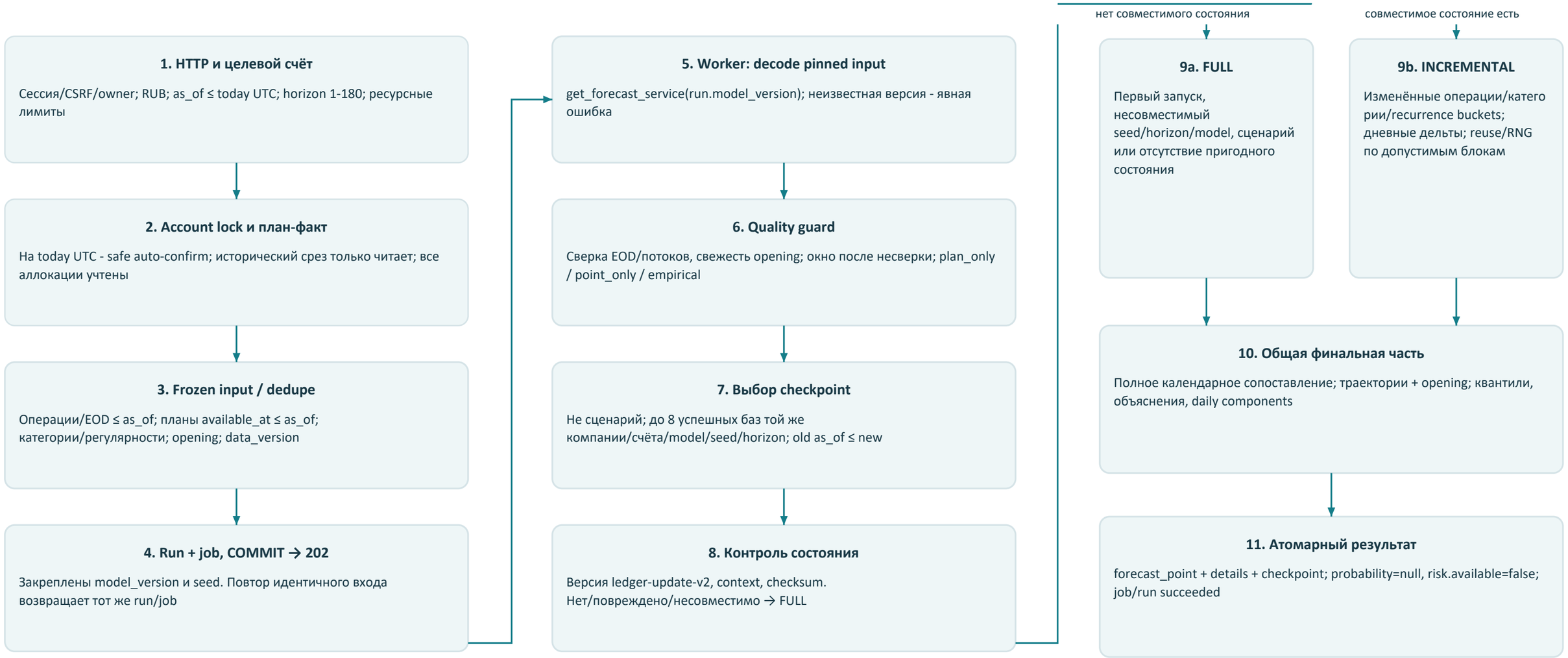


Развилки и ошибки сохранены

Ошибки preview исправляются до commit. Конфликт операции или документа откатывает весь пакет. Идентичный commit возвращает прежнюю сводку. Исторически неоднозначное состояние плана → 409; превышение бюджета → 422 без записей. Commit импорта не enqueue прогноз: обычный кабинет отправляет отдельный POST после успеха.

A05. Прогноз: снимок, качество, full и incremental

Источники: api/v1/forecasts.py; forecasting/snapshot.py; v2/service.py, quality.py, incremental_*; jobs/incremental.py.



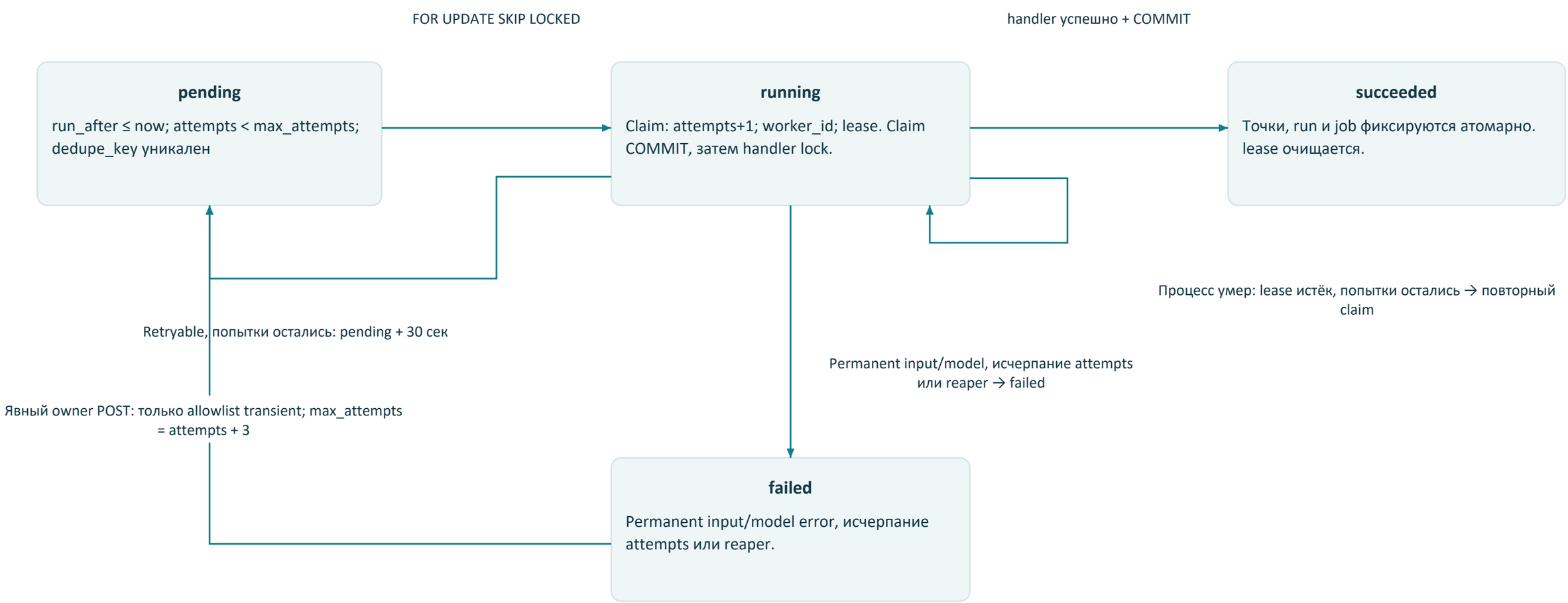
Совместимость и ограничения

Новые факты, поздние факты, категории, планы и движение даты вперёд могут использовать incremental. При смене as_of пересматриваются группы; при смене состава bootstrap-блоков сетка создаётся заново. Сценарии - полный расчёт. API по-прежнему готовит полный snapshot: весь запрос не объявляется O(delta). Старый успешный результат не переписывается.

Итоговая диагностика: calculation.mode, reason, source_forecast_id, categorized/reused_transactions, recurrence_buckets_recomputed, daily_aggregates_changed, trajectory_values_updated и bootstrap_paths_generated.

A06. Очередь: все статусы и ветви восстановления

Источники: jobs/queue.py, worker.py, recovery.py и handlers.py. Внешние побочные эффекты не покрываются транзакцией PostgreSQL.



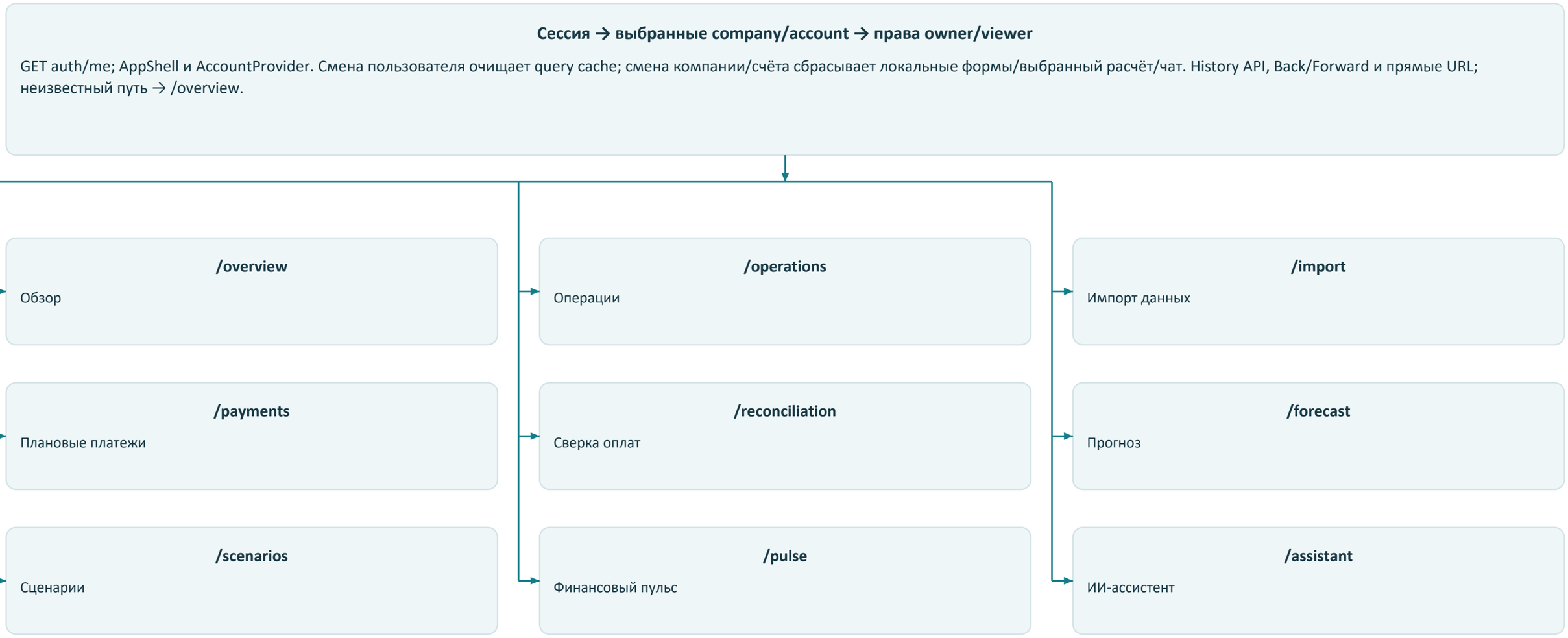
Дополнительные обязательные условия

Живой handler удерживает row lock, поэтому истечение lease само по себе не дублирует исполнение. Старый worker сверяет locked_by и attempt до записи. Reaper (до 100 строк, SKIP LOCKED) закрывает running с истёкшим lease и исчерпанными attempts; код lease_expired_attempts_exhausted. ForecastRun.failed/finished_at синхронизируются; успешный результат не стирается.

Повтор POST не возобновляет ошибки данных, неизвестной модели и произвольные неклассифицированные ошибки. IDs, input и seed сохраняются; номер attempts не сбрасывается. Lease по умолчанию 300 сек, исходный max_attempts 3, poll 1 сек.

A07. Кабинет: все маршруты и автоматические обновления

Источники: frontend/src/app/router.tsx, AppShell.tsx, account-context.tsx, forecast-controller.tsx; features/assistant/chat.ts.



Успешное действие обычного кабинета	Срез нового POST forecast	Поведение
Commit выписки	preview.period_end	Новый обычный базовый прогноз
План / обязательство / сверка	today UTC	Новый обычный базовый прогноз
Ручная категория	Текущий выбранный as_of	Новый обычный базовый прогноз
History / reload / смена даты / раздела	Автоматического POST нет	Чтение; дата требует явного запуска
Действие ассистента	Собственное явное подтверждение	Запись помечает график устаревшим; propose_forecast открывает созданный job

Viewer не запускает автоматическую запись. Poll job каждые 1500 мс до succeeded/failed; поздние ответы отбрасываются; terminal failed не порождает бесконечные POST. Никакой push-подписки на изменения стороннего API-клиента нет.

A08. Сохранённый результат: сценарии, разрывы и пульс

Источники: api/v1/scenarios в forecasts.py; cash_gaps/service.py; pulse/frozen.py, service.py. Все ветви сохраняют исходную базу.

Успешный frozen forecast_run + input_snapshot + forecast_point

Старые записи без snapshot читаются с ограничениями; для зависимых вычислений требуется новая база.

Обычный сценарий: POST scenarios

1-50 changes; отдельный job/run; та же закреплённая модель; сценарий считается full

Все четыре типа

delay_event; reduce_irregular_inflow; add_expense; increase_recurring

GET comparison

Сохранённые base/scenario; проверка даты, горизонта, model/data/seed. Другой seed исключает paired comparison

GET cash-gaps

Opening на as_of + дневные p50; existing/new эпизоды; ноль - восстановление; причины минимума

POST cash-gap-preview

1-3 действия: delay или financing. Перенос закупки требует согласия; защищённые расходы не переносятся

Ответ без записи

Медианная гипотеза; стоимость/возврат/долг за горизонтом; нет job и новой вероятности; viewer + CSRF разрешены

GET pulse

Точная сохранённая p50-кривая и frozen вход, не live-таблицы; buffer_minor опционален, 0 допустим

Все продуктовые направления

Касса/подушка/действия; аудит расходов; B2B-концентрация и клиентский stress

Гипотезы с ограничениями

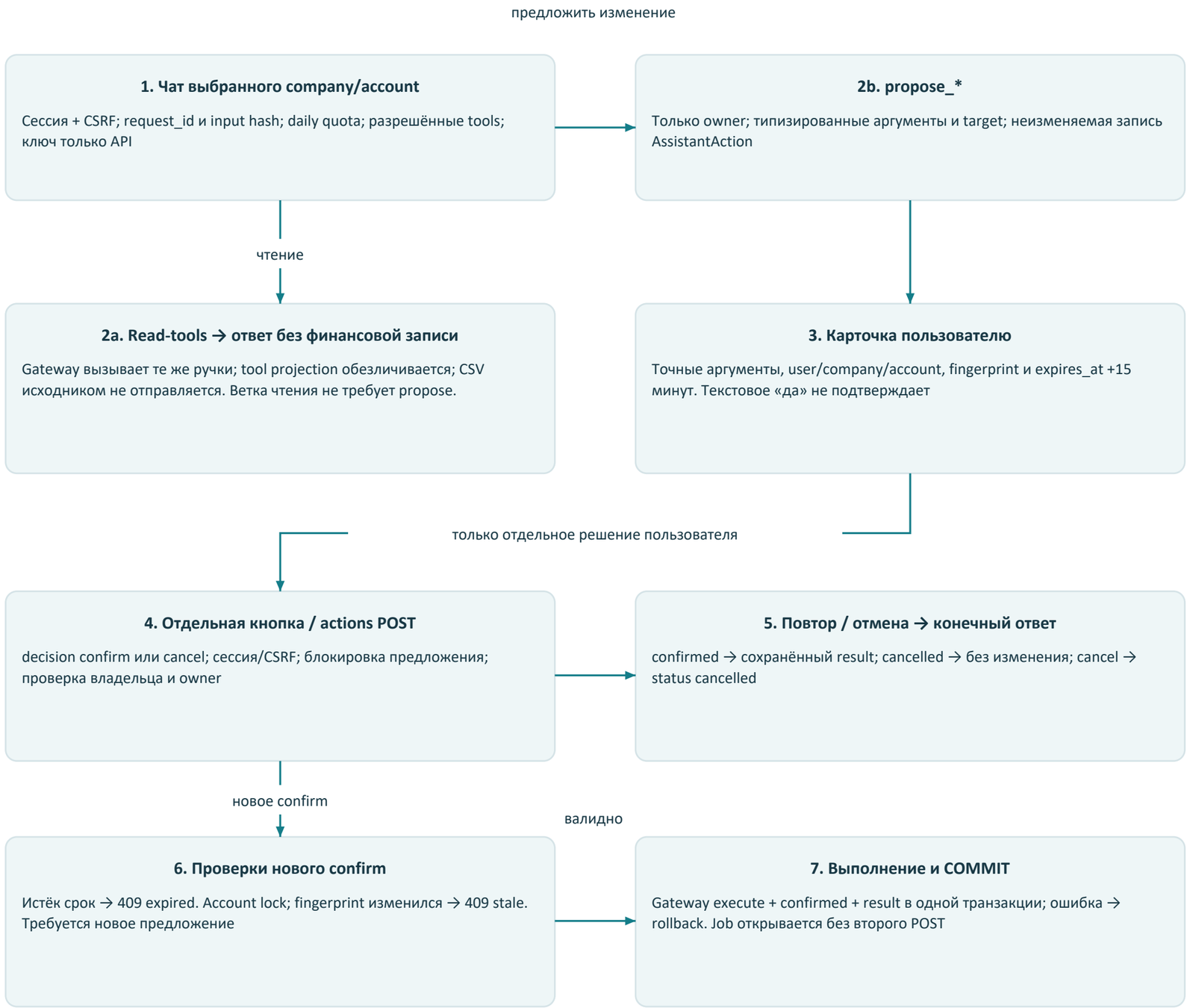
Техническая поправка к p50 не является выручкой; peers до 30.09.2026 отключены; экономии пересекаются

Общие границы

Факты и реальные планы не меняются. Числовая вероятность нового разрыва недоступна: null не равен 0%. Финансирование - сценарий с раскрытой стоимостью, не предложение банка. Риск дневной, внутридневная ликвидность не определяется. Подходящая рекомендация пульса не означает согласие контрагента на отсрочку.

A09. Ассистент: чтение, предложение и явное решение

Источники: modules/assistant/contracts.py, service.py, gateway.py, actions.py; api/v1/assistant.py. Право подтверждения модели не передаётся.



Все восемь записывающих предложений

1. propose_forecast
2. propose_scenario
3. propose_create_plan
4. propose_update_plan
5. propose_category
6. propose_reconcile
7. propose_match
8. propose_commit_import

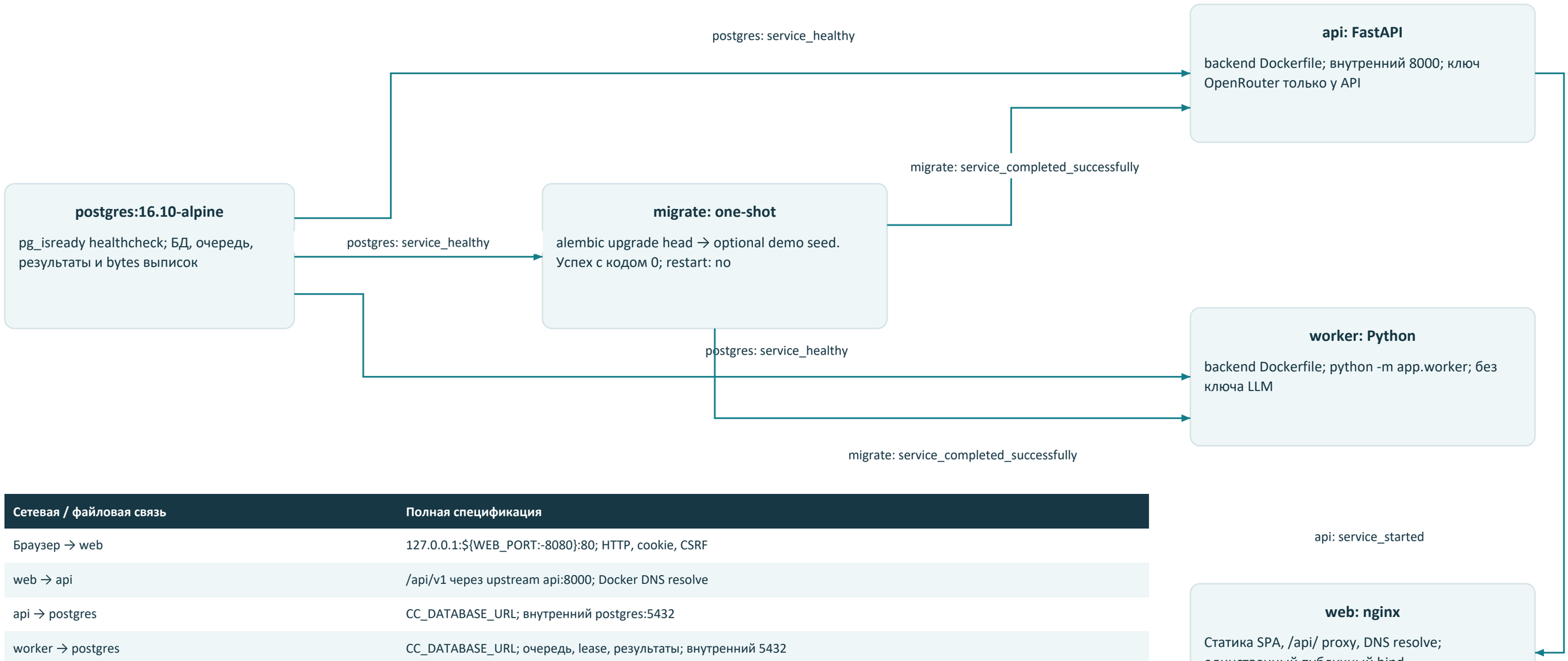
Другие разрешённые пути

GET assistant/status; POST assistant/chat; POST assistant/uploads (preview); POST actions/{id}. Viewer может читать, но не предлагать и не подтверждать финансовую запись. Произвольные SQL, URL и исполнение кода модели недоступны.

Свободный текст отправляется провайдеру. Очистка видимой истории не удаляет серверные turn/action. Числа берутся из серверных данных; модель не подтверждает действия.

A10. Compose: все зависимости запуска, сеть и хранение

Источник: infra/docker-compose.yml, Dockerfile, nginx.conf, migrate_and_seed.sh. Сплошные стрелки - все шесть depends_on; сеть дана отдельной полной таблицей.



Сетевая / файловая связь	Полная спецификация
Браузер → web	127.0.0.1:${WEB_PORT:-8080}:80; HTTP, cookie, CSRF
web → api	/api/v1 через upstream api:8000; Docker DNS resolve
api → postgres	CC_DATABASE_URL; внутренний postgres:5432
worker → postgres	CC_DATABASE_URL; очередь, lease, результаты; внутренний 5432
migrate → postgres	CC_DATABASE_URL; Alembic и demo seed; внутренний 5432
api → OpenRouter / DeepSeek	Необязательный HTTPS-чат; секрет не передаётся worker/frontend
postgres → pgdata	/var/lib/postgresql/data; постоянный volume; down -v удаляет данные

Restart и учётные данные

postgres, api, worker, web: unless-stopped. API и PostgreSQL не публикуют порты наружу. Seed создаёт отсутствующего пользователя; изменение CC_DEMO_PASSWORD не меняет пароль уже созданной записи.